Large Neighborhood Search for Capacitated Facility Location with Customer Incompatibilities

Ida Gjergji
DBAI, TU Wien
Vienna, Austria
ida.gjergji@tuwien.ac.at

Lucas Kletzander
DBAI, TU Wien
Vienna, Austria
lucas.kletzander@tuwien.ac.at

Nysret Musliu
DBAI, TU Wien
Vienna, Austria
nysret.musliu@tuwien.ac.at

Andrea Schaerf
University of Udine
Udine, Italy
andrea.schaerf@uniud.it

Abstract

A new variant of the classic capacitated facility location problem, which considers incompatibilities between customers, has recently been introduced in the literature. This problem captures the situation where given pairs of customers cannot be served by the same facility. Such a feature is crucial for many practical cases of location problems, such as the presence of hazardous or polluting materials or contention between competing costumers. In this paper, we propose a Large Neighborhood Search (LNS) method to solve this problem. Within the framework of LNS, we introduce three different destroy operators and we use an exact solver in the repair phase. We critically analyze the effectiveness and the efficiency of both destroy and repair operators. The experimental analysis shows that our new method outperforms existing state-of-the-art metaheuristics, providing new best solutions for all available benchmark instances.

CCS Concepts

• **Computing methodologies** → Search methodologies; • **Mathematics of computing** → Combinatorial optimization; • **Theory of computation** → Algorithm design techniques.

Keywords

Large Neighborhood Search, Facility Location Problem, Discrete Optimization

ACM Reference Format:
Ida Gjergji, Lucas Kletzander, Nysret Musliu, and Andrea Schaerf. 2025. Large Neighborhood Search for Capacitated Facility Location with Customer Incompatibilities. In *Genetic and Evolutionary Computation Conference (GECCO '25), July 14–18, 2025, Málaga, Spain.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3712256.3726355

1 Introduction

Facility location problems deal with determining optimal locations for facilities such as warehouses, factories, distribution centers, or service centers to meet the demands of customers or clients. These problems arise in various fields including logistics, supply chain management, telecommunications, and public services planning. The capacitated facility location problem (CFLP) extends the basic facility location problem [9], which is an NP-hard problem [5], by adding capacity constraints to facilities. The goal is to determine the optimal location of facilities while ensuring that the total quantity of goods served by each facility does not exceed its capacity.

In the *multi-source* version of the capacitated problem (MS-CFLP) each customer can be served by a set of different facilities in order to collect its required quantity of goods. The alternative is the *single-source* version, in which each customer is served entirely by a single facility. The *multi-source capacitated facility location problem with customer incompatibilities* (MS-CFLP-CI) has been recently proposed by Maia *et al* [12], as an extension of MS-CFLP, in which there are pairs of customers that cannot be served by the same facility. This additional constraint may arise in situations where a customer does not want to share the supplying place with its competitors. Another application comes from materials subject to specific regulations or legal requirements that necessitate separate handling. Two challenging datasets have been proposed by Maia *et al* [12] and by Ceschia and Schaerf [4], respectively. The instances of these datasets range from relatively small instances, with 50 facilities, to huge ones, up to 3000 facilities; and only the smaller ones, up to 150 facilities, have been solved to optimality.

Although advanced metaheuristic approaches [4, 12] were applied to these datasets, there is still room for improvement. For example, the optimal values, for instances up to 150 facilities, are not matched yet by the metaheuristic methods.

In this work, we propose a novel solution approach for the MS-CFLP-CI based on Large Neighborhood Search (LNS). Our approach includes novel destroy operators and a repair operator that relies on an exact Mixed Integer Programming (MIP) solver.

Our method, properly tuned in a statistically-principled manner, improves the state-of-the-art results for all instances of both available datasets. Summarizing, the main contributions of our paper are:

- We investigate for the first time a Large Neighborhood Search approach for this problem.
- We propose three novel destroy operators, such that their combination has been able to achieving state-of-the-art results.

This work is licensed under a Creative Commons Attribution 4.0 International License.
GECCO '25, July 14–18, 2025, Málaga, Spain
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1465-8/2025/07
https://doi.org/10.1145/3712256.3726355

- We make use of modern tuning methods and deeply analyze our approach including the impact of operators.
- We tested our method on both publicly available datasets for the problem, which contain very large and challenging instances (up to 3000 facilities and 8000 customers). Our experiments show that our approach outperforms previous methods with statistical significance, by improving upper bounds for all existing instances in the literature.

The paper is organized as follows: in Section 2 we give a formal definition of the MS-CFLP-CI problem. In Section 3 we discuss related work. Then, in Section 4 we describe the components of our LNS, which are the initial solution and the destroy and repair operators. After analyzing the effectiveness of the destroy operators and the parameter configuration of LNS, in Section 5 we present the experimental results where we compare with state-of-the-art methods. Finally, in Section 6 we conclude our work and present ideas for future research.

2 Problem Definition

In the MS-CFLP-CI, there are n customers to be served, each one with an associated demand d_i $(i = 1, \dots, n)$ to be satisfied. There are m facilities that can be opened, each one with a capacity s_j and an opening cost f_j $(j = 1, \dots, m)$. The cost of shipping one unit of good from facility j to customer i is denoted by c_{ij}. Finally, there is a set of pairs of incompatible customers $\mathcal{I}$, such that for each $\langle i_1, i_2 \rangle \in \mathcal{I}$, we state that i_1 and i_2 cannot be served by the same facility.

The problem consists in deciding which facilities to open and the quantities shipped to each customer from each (open) facility, in such a way that the constraints on the capacity of the facilities are satisfied, the demands of the customers are fully met, and the incompatibilities are respected. The objective function to be minimized is the sum of the opening costs and the shipping costs.

The mathematical model for the MS-CFLP-CI has been developed by Ceschia and Schaerf [4], reported here for self-containedness.

$$\min z = \sum_{i=1}^{n} \sum_{j=1}^{m} c_{ij} x_{ij} d_i + \sum_{j=1}^{m} f_j y_j \tag{1}$$

$$\sum_{j=1}^{m} x_{ij} = 1, \quad i = 1, \dots, n \tag{2}$$

$$\sum_{i=1}^{n} d_i x_{ij} \le s_j y_j, \quad j = 1, \dots, m \tag{3}$$

$$x_{ij} \le w_{ij}, \quad i = 1, \dots, n;\ j = 1, \dots, m \tag{4}$$

$$w_{ij} \le y_j, \quad i = 1, \dots, n;\ j = 1, \dots, m \tag{5}$$

$$w_{i_1 j} + w_{i_2 j} \le 1, \quad \langle i_1, i_2 \rangle \in \mathcal{I};\ j = 1, \dots, m \tag{6}$$

$$y_j \in \{0, 1\}, \quad j = 1, \dots, m \tag{7}$$

$$x_{ij} \in [0, 1], \quad i = 1, \dots, n;\ j = 1, \dots, m \tag{8}$$

$$w_{ij} \in \{0, 1\}, \quad i = 1, \dots, n;\ j = 1, \dots, m \tag{9}$$

The decision variable y_j takes value 1 if facility j is open, and 0 otherwise, while the decision variable x_{ij} represents the fraction of the demand of customer i supplied by facility j. The binary decision variables w_{ij} are introduced to manage the incompatibility constraints, so that w_{ij} is equal to 1 if the customer i is supplied by facility j (even partially), 0 otherwise. Constraints (6) express the incompatibilities, whereas Constraints (4) link variables w_{ij} to variables x_{ij}.

Notice that Constraints (5) are redundant, as the rule that only open facilities can supply goods is already enforced by Constraints (3). They are customarily added given that they normally speed up convergence, by providing better cuts.

As shown by Ceschia and Schaerf [4], the presence of incompatibilities makes the problem more complex in terms of both the computational time to solve it and the quality of the solutions. In fact, due to incompatibilities, the subproblem of finding the optimal supply from the open facilities is already NP-hard [7], whereas it is solvable in polynomial time in the basic case.

3 Related Work

The literature on facility location is extremely vast. We refer to the book by Laporte *et al* [9] for a general and comprehensive introduction to the topic.

Restricting to the multi-source capacitated version of the problem (MS-CFLP), the most known and challenging benchmark is dataset `C` introduced by Guastaroba and Speranza [8], with instances of 2000 facilities. The best known results on this dataset have been obtained by Avella *et al* [2] using flow techniques with an average gap from the lower bound of less than 0.1%, improving on the previous best ones of the Benders decomposition by Fischetti *et al* [6], which were about 1% above on the most difficult instances.

Moving to our specific version of the problem (MS-CFLP-CI), as already mentioned, it has been proposed by Maia *et al* [12], who also provided a dataset of 30 instances, with size ranging from 50 to 3000 facilities, called `wlp` and available online[1]. They propose four alternative metaheuristic approaches and compare them using two different time limits. In detail, the time limits proposed are based on the number of facilities m, and they are set to $10\sqrt{m}$ and m, respectively. All their methods were run on the same PC in order to have a fair comparison on a common ground. The method that turned out to be the best one is MineReduce, an approach based on data mining and iterated local search [11]. The MS-CFLP-CI has been subsequently dealt with by Ceschia and Schaerf [4] using a Simulated Annealing approach. The key features of their method are the two-source restriction (i.e., maximum two suppliers per customer) and a pre-processing procedure that reduces the search space by restricting, based on the shipping costs, to a small subset of possible suppliers for each customer. The Simulated Annealing of Ceschia and Schaerf improved upon MineReduce of [12] on all instances, except for the smallest ones, with an average improvement of about 6%. Using the model shown in Section 2, Ceschia and Schaerf [4] computed the lower bounds for dataset `wlp`. However, they have been able to obtain them only for instances up to 400 facilities, and strict ones (below 1% gap) only up to 200 facilities. Finally, Ceschia and Schaerf also proposed a new dataset, called `cflp-ci` and available online[2], composed of 50 instances whose sizes range between 69 and 2946 facilities; and they provide their results on this dataset for both timeouts.

[1] https://github.com/MESS-2020-1/Instances
[2] https://github.com/iolab-uniud/ms-cflp-ci

Algorithm 1 LNS framework

Input: Solution s, destroy methods $d_i(\cdot) \in \mathcal{D}$, repair methods $r_j(\cdot) \in \mathcal{R}$, acceptance criterion $a(\cdot, \cdot)$
Output: Improved solution s^*

1: $s^* \leftarrow s$
2: **while** a stopping criterion is not reached **do**
3: Select d_i from $\mathcal{D}$ and r_j from $\mathcal{R}$
4: $s' \leftarrow r_j(d_i(s))$
5: **if** $a(s, s')$ **then**
6: $s \leftarrow s'$
7: **end if**
8: **if** s' better than s^* **then**
9: $s^* \leftarrow s'$
10: **end if**
11: **end while**
12: **return** solution s^*

4 Large Neighborhood Search for MS-CFLP-CI

Local search is an algorithmic paradigm based on the simple idea of traversing the search space by iteratively stepping from one state to one of its neighbors. Large Neighborhood Search (LNS) is a local search metaheuristic based on the idea of defining the neighborhood by means of one or more *destroy* and *repair* operators.

LNS has been originally proposed by Shaw [16], although similar approaches have also been considered independently (see the survey by Ahuja *et al* [1] for alternative proposals). A comprehensive introduction to LNS has been provided by Pisinger and Ropke [14].

The framework of LNS is given in Algorithm 1. In line 4, a destroy operator shreds a part of the current solution and then a repair operator rebuilds it in a new way. In case of more than one operator of any kind, a strategy is needed for the selection of the operator in a given iteration in line 3. The destroy operators are normally stochastic, whereas the repair operators may also be deterministic. The most common acceptance criterion, applied in line 5, is accepting a solution if it is better than the current solution. When the acceptance criterion is fulfilled, s' is accepted as the current working solution s in line 6, while the best overall solution is updated in line 9.

4.1 Solution Representation

For Large Neighborhood Search, the solution is represented by a matrix that stores the quantity of goods shipped from each facility to each customer. We consider only solutions that satisfy the constraints:

- Goods are shipped only from open facilities.
- The demand of the customers is fully satisfied.
- The capacity of the facilities is never exceeded.
- Incompatible customers are not served by the same facility.

In order to speed up the search process, we make use of several auxiliary data structures. First, we have a set of open facilities F and a set of closed facilities G. For any open facility $f \in F$, we have a list C_f that contains all the customers assigned to this facility. On the other hand, for any customer i we have a list a_i with its assigned facilities and a second list q_i indicating the corresponding quantity from these facilities.

4.2 Initial Solution

We propose the following initial solution heuristic for the MS-CFLP-CI to provide a good starting solution to LNS. First, we sort the facilities in increasing order of their opening cost. Based on the cumulative capacity, we select enough facilities to fulfill the demand of the customers. As we have incompatibility constraints, a set of k additional facilities is needed for the overall assignment. We checked the feasibility of the instances and initial cost for different values of k. For $k \in \{0, 1, 2, 3, 4, 5\}$ results are shown in Table 1. For $k = 0$ and $k = 1$ infeasible solutions are reported. From the obtained results, $k = 5$ provides the lowest total average cost for all instances without affecting the runtime.

Additionally, we investigate the order of assigning the customers to the open facilities. There are three strategies that we consider: uniformly, based on decreasing order of demand, and based on regret values. The notion of regret values is defined by Mulvey and Beck [13] as the absolute difference between the two closest open facilities for any customer i. In our case, we compute the regret values as the absolute difference between the two cheapest open facilities in terms of unit cost for all customers. The results show that the fastest performing strategy is the uniform one, while there are no significant differences in terms of total average cost among the three of them. For any customer i, the open facilities are sorted in increasing order of unit cost. Then the assigned quantity from the customer i to the facility is the minimum between customer's demand and facility's capacity. We go through the sorted facilities until the demand of every customer has been met. This procedure is deterministic and we run it only one time per instance.

4.3 Destroy Operators

A destroy operator defines a part of the solution s which is subject to changes in an iteration, while the other portion of the solution remains intact. We refer to this part as the *sub-problem*. Initially, we considered operators from the literature defined for other versions of the facility location problem like random selection of facilities, swap operator (where facilities remain open but only re-assignment of customers, whenever possible, is allowed), high cost facilities (where facilities with higher cost are prioritized), and underloaded facilities (where facilities with higher unused capacities are prioritized). However, these operators did not yield good results. We believe this is because of the incompatibility constraints. This comprehensive investigation led to the definition of novel operators specifically designed for the MS-CFLP-CI.

In our case, the *sub-problem* is defined in terms of the group of customers, the group of open facilities, and the group of closed facilities that are involved. In order to effectively define these elements of the sub-problem, we investigate the relationships between open facilities – closed facilities, open facilities – customers, and closed facilities – customers. Note that the most important feature when we consider only customers is whether or not pairs of them are incompatible. In the repair phase, those customers can be reassigned to other facilities without affecting the feasibility of the solution as the incompatibilities are integrated in the constraints of the mathematical model used for the sub-problem.

The three destroy operators proposed for the LNS are as follows, using a size parameter ℓ:

Table 1: Average total cost and time (in seconds) for different values of k

	k=2		k=3		k=4		k=5	
	cost	time (s)	cost	time (s)	cost	time (s)	cost	time (s)
avg	704094.99	5.25	701488.79	5.28	699128.88	5.27	**696518.04**	5.26

- `Cheapest facilities (CW)`: For a randomly selected facility $f \in F$, identify its customers C_f. For any open facility f', we define a measure of a quality $Q_{f'}$ as:

$$Q_{f'} = \frac{\sum_{i \in C_f} c_{if'}}{|C_f|} \tag{10}$$

  $Q_{f'}$ gives the average shipping cost of a facility for the set of customers C_f. The idea is to get the facilities that would in average provide service in a lower cost to the customers in C_f. The ℓ facilities with lowest $Q_{f'}$ and their customers would be the elements of a sub-problem together with the initially selected facility f and its customers C_f.
- `Cheapest customers (CC)`: For a randomly selected facility f, identify its customers C_f. Additionally, identify ℓ customers that have the lowest unit cost with the facility f. In the sub-problem we would then have facility f and its customers C_f joined with the ℓ chosen customers and the facilities to which they are currently assigned.
- `Expensive customers (EC)`: For all open facilities we identify the most expensive customer based on the cost incurred by each customer. The pool of expensive customers contains the most expensive customer per facility. In any iteration, we either pick randomly one customer from this pool or we choose the most expensive customer from this pool. Consequently, for the selected customer we identify ℓ facilities that offer the lowest unit cost. These ℓ facilities and their customers together with the initially selected customer and its appointed facilities are the sub-problem elements.

Another component of the sub-problem is the group of closed facilities. Preliminary results show that it is favorable to also have closed facilities using the same parameter ℓ in each iteration so as to check whether they are better candidates for the group of selected customers compared to the involved open facilities. Additionally, as the quality measure in Equation (10) has shown beneficial impact, we also use it for defining one of the techniques for the closed facilities, to ensure good results with all destroy operators. There are two strategies that we consider for the collection of the closed facilities, which are chosen randomly according to a probability parameter p_{MC}:

- `Random facilities(R)`: Select ℓ random closed facilities from G.
- `Minimum cost facilities (MC)`: Get ℓ closed facilities that offer minimum Q_g, where Q_g is calculated for any closed facility $g \in G$ as in Equation (10) considering all customers of the defined sub-problem.

4.4 Repair Operator

In the repair phase, the destroyed part of the solution is restored. To achieve this, we declare the model as given in Section 2 for the sub-problem using the MIP solver Gurobi. For the components of the sub-problem all changes are allowed: open facilities can remain open or get closed, closed facilities can remain closed or be opened, and customers are allowed to change their previously assigned facilities. The sub-problem requirements are the same as for the master problem.

A crucial element of integrating an exact solver in the repair stage is the time it needs for an iteration. Declaring the model and solving it in an iterative fashion can be extremely time consuming. To avoid this, we carefully examine the possible options to provide a more compact model. Regarding this matter, we add one constraint that limits the number of facilities that can be open at the end of an iteration. If there are o open facilities involved in the given iteration, then the extra constraint would allow at most $o + 2$ open facilities after the repair stage. From initial results, we could see that usually no more than 2 additional facilities are opened. Constraining on the number of possible facilities to open is beneficial in terms of the runtime that the repair stage takes.

Furthermore, at each step we provide to Gurobi a *cutoff* value, which is an upper limit for the solution cost. The cutoff value is the initial cost of the sub-problem that is considered in the iteration. This action helps to speed up the search process.

5 Evaluation

Previous results by both Maia *et al* [12] and Ceschia and Schaerf [4], which represent state-of-the-art methods for MS-CFLP-CI, have been obtained on an AMD Ryzen ThreadripperPRO 3975WX with 32 cores (3.50 GHz), with 64 GB of memory and running Ubuntu Linux 22.4, on a single core. For a direct comparison, we were able to run the LNS experiments on the same machine.

The proposed LNS is compared with other methods based on the percentage gap value defined as follows:

$$\text{GAP}_{\text{sol}} = \frac{(Z_{sol} - \text{BKS})}{\text{BKS}} \cdot 100 \tag{11}$$

In Equation (11), Z_{sol} is the objective function value of the solution, and BKS represents the best-known solution reported in the literature.

The methods compared with LNS are all methods from Maia *et al* [12] and Ceschia and Schaerf [4]: MineReduce-based Multi-Start Iterated Local Search (MR-MS-ILS), Greedy Randomized Adaptive Search Procedure (GRASP), Permutation-coded Evolutionary Algorithm (PcEA), Multi-start Greedy (MG), and Simulated Annealing (SA).

In the experimental evaluation we consider the dataset `wlp` of Maia *et al* [12] and the dataset `cflp-ci` by Ceschia and Schaerf

Table 2: Results from CPLEX and Gurobi

Instance	m	n	CPLEX	Gurobi
wlp01	50	115	**28716**	**28716**
wlp02	100	253	**52952**	**52952**
wlp03	150	345	**64296**	**64296**
wlp04	200	479	84633	84633
wlp05	250	601	107323	103857
wlp06	300	705	115295	111654
wlp07	400	1012	170100	162277
wlp08	500	1277	–	187938
wlp21	75	172	**38067**	**38067**
wlp22	175	428	74473	74469
wlp23	275	694	124991	119091
wlp24	450	1128	176721	168816
cflp-ci-11	69	156		**30728**
cflp-ci-39	383	1131		189284

[4], with a total of 80 instances. The BKS for 78 out of 80 instances are achieved from Simulated Annealing by Ceschia and Schaerf [4], and the other 2 by MineReduce [12].

As our LNS method is not deterministic, we report the results of 10 runs with different seeds, which are recorded for reproducibility purposes.

5.1 Exact Approach

Before investigating LNS for the MS-CFLP-CI problem, we verified whether we could solve the problem by using Gurobi directly on the mathematical model given in Section 2 for all available instances. Similar analysis was done by Ceschia and Schaerf [4] using CPLEX, but only for dataset `wlp`. The time budget set for these experiments was 7200 seconds. Our results are shown in Table 2, where we compare the results from CPLEX by Ceschia and Schaerf [4] and our results using Gurobi (more details in the Appendix).

The entries in bold indicate the solutions that both CPLEX and Gurobi report as optimal. For other instances, each entry is the objective function value that the solver delivers at the end of the run. Note that for the majority of the instances no results are reported as they are beyond the capacities of exact solvers. This is due to their size that causes the execution to run into a memory error.

Regarding Table 2, the largest instance that Gurobi can solve to optimality has 150 facilities, and it can only give feasible solutions for instances with no more than 500 facilities with a runtime of 7200 seconds. Moreover, for the instances that it can handle, Gurobi gives better bounds for 7 instances compared to CPLEX.

As for the MS-CFLP-CI problem we tackle instances with a number of facilities up to 3000, the use of alternative methods such as metaheuristics is necessary to tackle such large instances.

5.2 Ablation Analysis

We investigate the performance of the designed destroy operators by performing an ablation analysis. We consider each destroy operator individually `CW`, `CC`, `EC`, combinations of them two by two `CW+CC`, `CW+EC`, `CC+EC`, and lastly all of them `CW+CC+EC`. For all

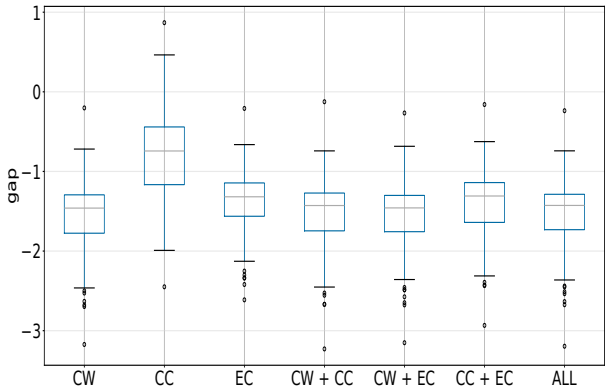

Figure 1: GAP for different destroy operators

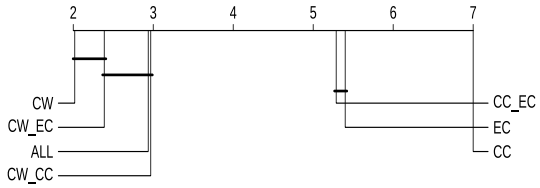

Figure 2: Critical difference plot for all the evaluated operator configurations

of these configurations the gap values from the average results of 10 runs are shown in Figure 1.

The first observation from the ablation analysis is all configurations provide mostly negative gap values, which indicate that new solutions have been found. Actually, the operator `CC` seems to be performing the worst among other evaluations as only this configuration gives some positive gap values, but even in this case the clear majority is negative, showing very good performance compared to state of the art.

In order to better assess the performance of each configuration, we also perform statistical tests using the R script `scmamp` [3]. Based on the average results for these 7 configurations, the Friedman test gives a p-value smaller than 2.2×10^{-16} which shows that not all of them have the same performance. Furthermore, we apply the Nemenyi post-hoc test which gives the ranking of the operators based on their average performance. According to these results, the configurations are ranked from the best to the worst performing configuration in the subsequent manner: `CW`, `CW+EC`, `CW+CC+EC`, `CW+CC`, `CC+EC`, `EC`, and `CC`. Operator `CW` appears in the best ranked positions, alone and in combination with other operators. A possible explanation for why such sub-problem composition is useful could be that having lowest cost facilities and their customers together allows for cheaper rearrangement of customers among facilities that would be harder in the presence of incompatibility constraints.

Figure 2 shows the results in a critical difference plot. Here, a lower index (given on top) shows better performance. Horizontal bars (e.g., between `CW+EC` and `CW`), indicate that no significant difference is given, while methods without a direct bar connection (e.g., `CW` and `ALL`) show significant difference.

Table 3: Parameter values used for the LNS

Parameter	Description	Domain	Value
v	sub-problem size	$[10, 200]$	50
p_{MC}	probability to use MC	$[0, 1]$	0.5
p_{CW}	probability to use CW	$[0, 1]$	0.9
t_g	time given to Gurobi (s)	$[10, 60]$	20

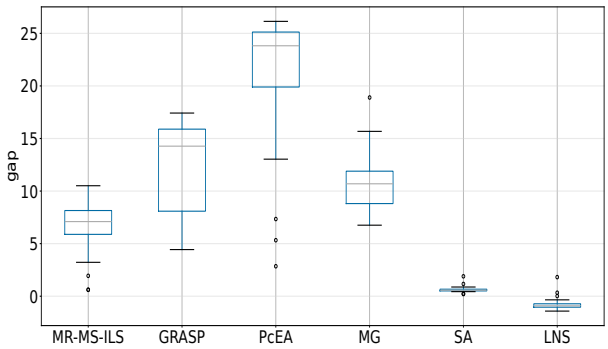

Figure 3: GAP values for dataset wlp, short timeout

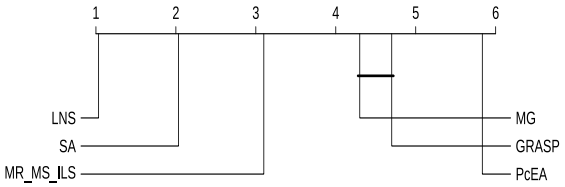

Figure 4: Critical difference plot for dataset wlp, short timeout

As no statistically significant difference is obtained between the performance of CW and CW + EC by the Nemenyi test, we run these two configurations again for half the time given in the initial experiments. Results show that CW + EC performs better than CW in this setting, so we use this configuration for the final experiments.

5.3 Parameter Configuration

The parameters considered for the LNS algorithm are the sub-problem size as v, the probability of selecting minimum cost closed facilities (MC) as p_{MC} (the probability of random selection of closed facilities (R) would be $1 - p_{MC}$), the time allocated to Gurobi in the repair stage as t_g, and the weights of the operators CW and EC as p_{CW} and p_{EC} (where $p_{EC} = 1 - p_{CW}$).

The time given to Gurobi t_g has been determined by manual trials. Based on the design of the repair phase, we fix this value to $t_g = 20$ seconds. For the other parameters, we use the automatic parameter configuration tool irace [10]. The instances used for the tuning stage are wlp01-wlp20. This subset of 20 instances was used during the tuning procedure in the methods introduced by Maia *et al* [12]. We use a budget of 2000 experiments in the tuning process with a timeout of 300 seconds per experiment.

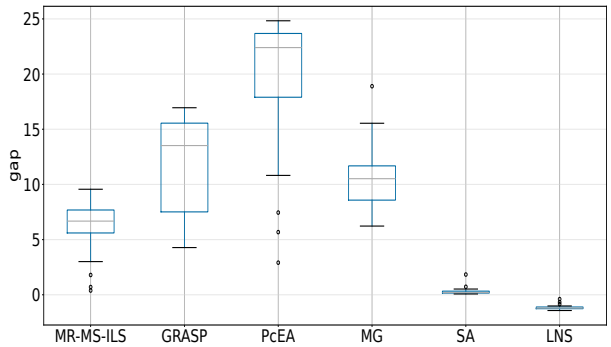

Figure 5: GAP values for dataset wlp, long timeout

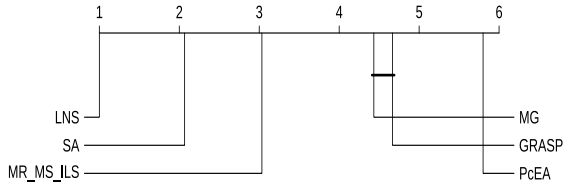

Figure 6: Critical difference plot for dataset wlp, long timeout

Based on manual trials, we defined values for the listed parameters which then were confirmed by irace. These values together with the ranges given to irace are displayed in Table 3. The sub-problem size is an essential parameter of the algorithm. Indeed, a high value of v can cause the repair stage to be really slow as it needs more time to declare and to solve a problem for more customers and facilities. On the other hand, a small value of v can lead to fewer and smaller improvements. Thus, it is mandatory to provide a value of v that works well for all of the instances. As recommended from irace we use $v = 50$ for our experiments. The destroy operator size ℓ is then calculated as defined by Equation (12), where we approximate the number of facilities that would offer service to v customers by using the ratio of the current number of open facilities to the number of customers:

$$\ell = \frac{v \cdot |F|}{n} \tag{12}$$

Other parameters are the weights of the operators which are used throughout LNS procedure for their selection based on the *roulette wheel* approach. The results from irace confirm using $p_{CW} = 0.9$ and $p_{EC} = 0.1$, in line with the ablation analysis results that CW is the most important operator.

5.4 Results for Dataset wlp

For dataset wlp we compare the results of MR-MS-ILS, GRASP, PcEA, MG, SA, and our proposed LNS. The experiments have been carried out for two different timeouts: $10\sqrt{m}$ and m.

From Figure 3, where the results for the shorter timeout are presented, the worst performing algorithm is PcEA. MR-MS-ILS, GRASP, and MG are also clearly outperformed by SA and LNS. These two methods show very good and very consistent results, with LNS further outperforming SA on almost all instances. The

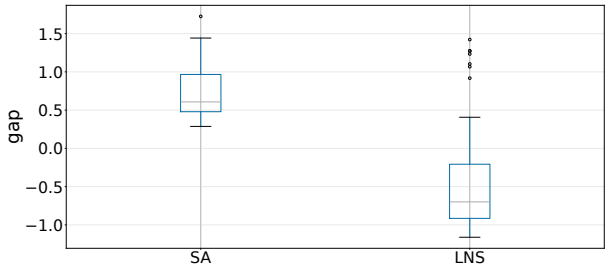

Figure 7: GAP values for dataset `cflp-ci`, short timeout

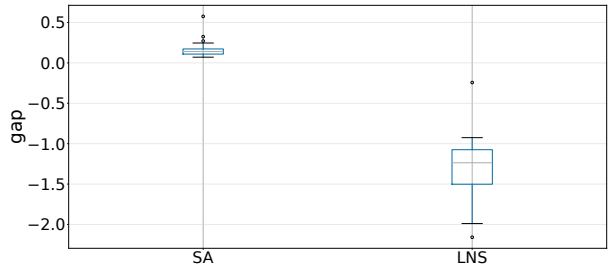

Figure 8: GAP values for dataset `cflp-ci`, long timeout

critical difference plot in Figure 4 confirms this picture, showing that LNS significantly improves results over SA, which is in turn significantly better than all other methods in comparison.

The situation is even clearer using the longer timeout demonstrated in Figure 5 . Now LNS consistently yields gap values below 0 for all instances, which indicate that new best solutions have been found.

Again, we perform statistical tests for all these methods based on the average reported results using `scmamp` [3]. The critical difference plot displayed in Figure 6 is very similar to the previous one, showing that LNS significantly outperforms all other methods.

We present additional data for comparing SA and LNS in Table 4, based on the long timeouts. We do not include the results from MR-MS-ILS, GRASP, PcEA, and MG as they are worse than SA (only MR-MS-ILS provides better results for 2 instances compared to SA). In Table 4, the first column shows the group of instances and the second column indicates the amount of entities involved in the respective group of instances. The number of entities is the sum of the number of facilities and the number of customers in an instance. The number of instances present in each group is given in the third column. Then, in the remaining columns we display the average results for each group of instances for the minimum and average costs obtained over 10 runs for both SA and LNS. For all the 4 subgroups of instances, LNS gives better results for the minimum and average values.

Actually, LNS finds new best solutions for all instances in dataset `wlp`. From the minimum values of LNS, across all instances, the minimum improvement in the gap value is −0.65% and the maximum improvement in the gap value is −1.65%.

5.5 Results for Dataset `cflp-ci`

Dataset `cflp-ci`, composed of 50 instances, was introduced by Ceschia and Schaerf [4], therefore only SA was tested previously; and so we only compare SA and LNS.

Again, in the shorter timeout as in Figure 7, LNS is better than SA for most instances. We perform Wilcoxon signed-rank test using the `scipy` module in python. With a significance level $\alpha = 0.05$, a p-value 3.13×10^{-10} is obtained. Such results confirm that LNS significantly outperforms SA. Again, the improvement is even stronger in Figure 8, where in the longer timeout LNS provides all gap values below 0.

Similar to results of dataset `wlp`, we also compare SA and LNS based on the average results of the grouped instances based on the number of entities. In Table 5, the first column shows the instance

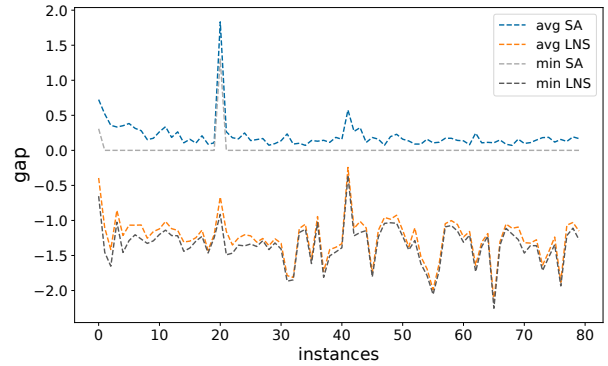

Figure 9: GAP values for all instances

group, the second column shows the number of entities in the corresponding group and the third column gives the number of instances in each group. Note that in dataset `cflp-ci`, there are more large instances. Again, LNS has a superior performance compared to SA in terms of both minimum and average results. LNS provides new best solutions for all the instances of this second dataset as well. Based on the minimum values gained from LNS, across all instances, the minimum improvement in the gap value is −0.36% and the maximum improvement in the gap value is −2.25%.

Moreover, we compare the performance of SA and LNS for the instances of the second dataset, long timeout by again performing Wilcoxon signed-rank test. With a significance level $\alpha = 0.05$ the obtained p-value 1.78×10^{-15} shows that LNS significantly outperforms SA.

5.6 Further Analysis and Discussion

Further details are given in Figure 9 to compare SA and LNS for all instances. The data of this graph represents gap values of the average and the minimum results from 10 runs of SA and LNS each, where the BKS are given from the minimum values obtained by SA (only 2 are from MR-MS-ILS). SA has slightly worse performance in small instances, and larger gaps between minimum and average in general. On the other hand, LNS seems to handle all of the instances well, which shows a very good performance.

In comparison, it seems that the advantage of LNS over SA grows for larger runtimes. Detailed results for dataset `wlp` and for dataset `cflp-ci` can be found in the Appendix. The neighborhoods used in the SA perform small changes iteratively. In contrast, LNS explores

Table 4: Comparing SA and LNS for dataset `wlp`

			SA		LNS	
group	entities	instances	min	avg	min	avg
1	< 1000	8	72026.88	72266.31	**70985.75**	**71149.49**
2	[1000, 3000)	9	215053.44	215532.24	**212274.00**	**212549.48**
3	[3000, 6000)	7	412187.71	412960.76	**406785.43**	**407147.37**
4	>= 6000	6	738703.00	739522.45	**728918.00**	**729322.03**

Table 5: Comparing SA and LNS for dataset `cflp-ci`

			SA		LNS	
group	entities	instances	min	avg	min	avg
1	< 3000	10	299350.90	299924.00	**295760.80**	**296125.25**
2	[3000, 6000)	11	567097.09	567913.34	**559273.64**	**559772.45**
3	[6000, 8000)	15	650426.40	651420.77	**640672.80**	**641136.89**
4	>= 8000	14	1119661.21	1120901.21	**1105322.71**	**1106034.87**

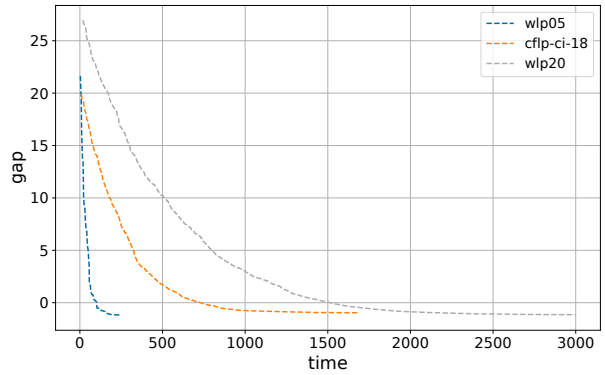

Figure 10: GAP values over time for 3 different instances

mid-sized neighborhoods. This could help to further improve solutions while SA has already reached a local optimum. Correctly setting the size parameter of approximately 50 customers per subproblem was essential for the high performance of LNS.

Another experiment that we conducted is the conversion of the proposed LNS into an Adaptive Large Neighborhood Search (ALNS) proposed by Ropke and Pisinger [15], where the weights of the operators are updated during the execution of the solver. As outlined by Turkeš *et al* [18] it is important to investigate if adding the adaptive layer to the LNS can further enhance the performance, although this doesn't always improve the results. From our preliminary experiments with ALNS, its performance was not superior to the ones of LNS (actually slightly worse), therefore we decided to remain with the simpler LNS version.

In Figure 10 we display the changes of the gap value over time for 3 instances with different sizes: `wlp05`, `cflp-ci-18`, and `wlp20`. Each of these instances has a distinct timeout, which is based on the corresponding number of facilities. Note that in the Figure 10 the threshold of a gap of 0 (the previous state-of-the-art solution) is reached after around or even slightly before half the given runtime.

Moreover, let us consider the data obtained from Gurobi in Table 2 for the smaller instances. Comparing the minimum gap values of LNS with respect to the bounds given from Gurobi, the maximum gap value is 0.52%, whilst the minimum one is 0.01%. Recall that the time given to Gurobi was 7200 seconds and LNS operates with m seconds time budget. This shows how efficient LNS is using the given runtime compared to the application of exact methods.

6 Conclusions

In this paper, we present a Large Neighborhood Search for the multi-source capacitated facility location problem with customer incompatibilities. The proposed method includes novel destruction operators and a repair step based on an exact solver that optimizes selected sub-problems in each iteration. Our work provides a detailed investigation of algorithm components, the configuration of our method, and analysis that includes statistical tests. Our experiments show that cheapest facilities `CW` and expensive customers `EC` operators are especially useful for obtaining good solutions. Furthermore, the repair operator has proven to be very effective, allowing the method to explore large neighborhoods in a short time. Overall, our evaluation demonstrates that our method currently represents the state of the art for this problem and can be used to solve large problems in a reasonable time.

A possible direction for future work is studying the possibility to hybridize Large Neighborhood Search with Simulated Annealing, for example along the lines proposed by Ropke and Pisinger [15]. Another possible direction would be to perform an Instance Space Analysis [17] in order to understand which are the features that make a given instance hard for each specific method. Finally, we plan to try to extend our method to other versions of the facility location problem, and compare with the state-of-the-art results for the available benchmarks for these variants.

Acknowledgments

This research was funded in whole by the Austrian Science Fund (FWF), project I 5443-N, Grant-DOI 10.55776. Our partners in the international project BioLOG are supported by the Polish National Center of Science (NCN) through grant DEC-2020/39/I/HS4/03533, and the Slovenian National Research Agency (ARRS) through grant N1-0223. This work is also partially funded by the Doctoral Program Vienna Graduate School on Computational Optimization, Austrian Science Foundation (FWF), under grant No.: W1260-N35 (https://vgsco.univie.ac.at/).

References

[1] Ravindra K Ahuja, Özlem Ergun, James B Orlin, and Abraham P Punnen. 2002. A survey of very large-scale neighborhood search techniques. *Discrete Applied Mathematics* 123, 1-3 (2002), 75–102.

[2] P. Avella, M. Boccia, S. Mattia, and F. Rossi. 2021. Weak flow cover inequalities for the capacitated facility location problem. *European Journal of Operational Research* 289, 2 (2021), 485–494.

[3] Borja Calvo and Guzmán Santafé Rodrigo. 2016. scmamp: Statistical comparison of multiple algorithms in multiple problems. *The R Journal, Vol. 8/1, Aug. 2016* (2016).

[4] Sara Ceschia and Andrea Schaerf. 2024. Multi-neighborhood simulated annealing for the capacitated facility location problem with customer incompatibilities. *Computers & Industrial Engineering* 188 (2024), 109858.

[5] Gérard Cornuéjols, George Nemhauser, and Laurence Wolsey. 1983. *The uncapicitated facility location problem.* Technical Report. Cornell University Operations Research and Industrial Engineering.

[6] Matteo Fischetti, Ivana Ljubić, and Markus Sinnl. 2016. Benders decomposition without separability: A computational study for capacitated facility location problems. *European Journal of Operational Research* 253, 3 (2016), 557–569.

[7] Dries Goossens and Frits CR Spieksma. 2009. The transportation problem with exclusionary side constraints. *4OR* 7 (2009), 51–60.

[8] Gianfranco Guastaroba and Maria Grazia Speranza. 2012. Kernel search for the capacitated facility location problem. *Journal of Heuristics* 18, 6 (2012), 877–917.

[9] Gilbert Laporte, Stefan Nickel, and Francisco Saldanha da Gama (Eds.). 2019. *Location science.* Springer International Publishing. doi:10.1007/978-3-030-32177-2

[10] Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. 2016. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives* 3 (2016), 43–58.

[11] Marcelo Rodrigues Maia, Alexandre Plastino, and Puca Huachi Vaz Penna. 2020. MineReduce: An approach based on data mining for problem size reduction. *Computers & Operations Research* 122 (2020), 104995.

[12] Marcelo Rodrigues Maia, Miguel Reula, Consuelo Parreño-Torres, Prem Prakash Vuppuluri, Alexandre Plastino, Uéverton S Souza, Sara Ceschia, Mario Pavone, and Andrea Schaerf. 2023. Metaheuristic techniques for the capacitated facility location problem with customer incompatibilities. *Soft Computing* 27, 8 (2023), 4685–4698.

[13] John M Mulvey and Michael P Beck. 1984. Solving capacitated clustering problems. *European Journal of Operational Research* 18, 3 (1984), 339–348.

[14] David Pisinger and Stefan Ropke. 2019. Large neighborhood search. *Handbook of metaheuristics* (2019), 99–127.

[15] Stefan Ropke and David Pisinger. 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation science* 40, 4 (2006), 455–472.

[16] Paul Shaw. 1998. Using constraint programming and local search methods to solve vehicle routing problems. In *International conference on principles and practice of constraint programming (CP-98).* Springer, 417–431.

[17] Kate Smith-Miles and Mario Andrés Muñoz. 2023. Instance space analysis for algorithm testing: Methodology and software tools. *Comput. Surveys* 55, 12 (2023), 1–31.

[18] Renata Turkeš, Kenneth Sörensen, and Lars Magnus Hvattum. 2021. Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search. *European Journal of Operational Research* 292, 2 (2021), 423–442.